

Details

Warehouse Line-Following & Inventory Robot



Internship Overview

This internship focuses on designing and developing an autonomous mobile robot for warehouse applications capable of following predefined paths and identifying inventory items.

The robot will use line-following techniques for navigation and integrate identification technologies such as RFID, QR codes, or barcodes for inventory logging.

This project introduces students to autonomous navigation, industrial automation concepts, and inventory management systems used in smart warehouses and logistics.

Project Aim

To develop a warehouse mobile robot where the system:

- Navigates autonomously using line-following
- Identifies inventory using RFID / QR / barcode systems
- Logs item data in real time

- Moves reliably along predefined warehouse paths
-

Technical Objectives & Learning Outcomes

During this internship, students will gain experience in:

- Line-following navigation
 - Sensor integration (IR / Vision)
 - RFID / QR / Barcode systems
 - Autonomous mobile robot control
 - Embedded programming
 - Data logging and automation
 - Industrial robotics concepts
-

Project Deliverables

By the end of the internship, students are expected to successfully demonstrate:

Autonomous Navigation

- Line-following implementation
 - Path tracking inside warehouse layout
 - Smooth motion control
-

Inventory Identification

- Integration of RFID / QR / Barcode module
 - Item detection and recognition
-

Data Logging System

- Recording inventory information
 - Storing item details
-

Onboard Control System

- Navigation logic implementation
 - Sensor-based decision making
-

System Integration & Testing

- Integration of navigation and inventory modules
 - Real-time testing in warehouse setup
-

Final Outcome

A fully functional warehouse robot capable of autonomously following predefined paths and identifying inventory items while recording data, demonstrating real-world concepts of industrial automation, logistics robotics, and smart inventory management.

Report

Project Progress Report
Warehouse Line-Following & Inventory Robot

Team Members

Rithesh R K

Ashwin Nair

Samruddhi R K

2nd Year

Department of Robotics & Automation

Under the Guidance of

Mr.Pulkit Sir

Duration of Project

02-02-2026 To 2-03-2026

Table of Content

CONTENT	PAGE NO.
Executive Summary	1
Technical Scope & Learning Objectives	1
System Architecture & Hardware Selection	2
Implementation Log & Milestone Tracking	3-4
Current Project Status	5
Challenges, Risks & Mitigation Strategies	5
Final Execution Plan & Timeline	6

1. Executive Summary

This report outlines the ongoing development of an autonomous mobile robot designed for smart warehouse management and industrial logistics. The project simulates a modern Industry 4.0 environment where autonomous systems navigate shop floors reliably while keeping real-time logs of inventory. **As of this report, a major project milestone has been successfully achieved: the mechanical assembly and chassis fabrication are 100% complete.** The system utilizes an Arduino UNO as its core processing unit, employing multi-array infrared (IR) sensors for precise line-following, ultrasonic sensors for collision avoidance, and RFID/vision systems for accurate inventory identification.

2. Technical Scope & Learning Objectives

The primary aim of this internship project is to bridge the gap between theoretical robotics and practical, real-world industrial automation.

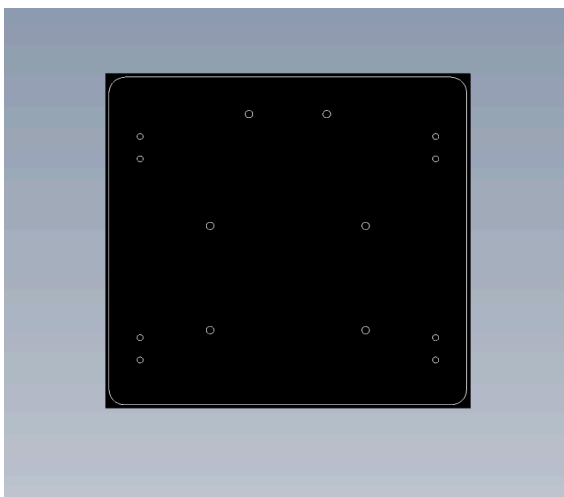
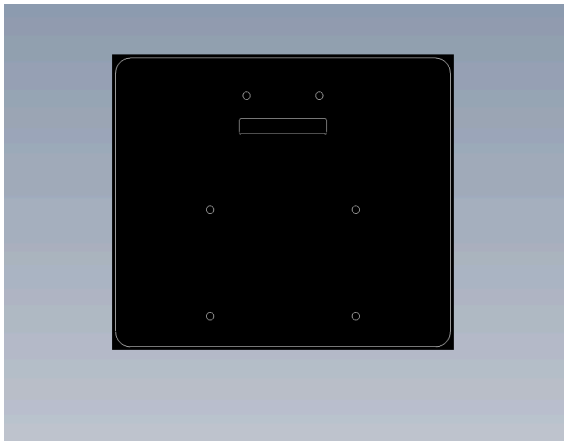
Key Technical Objectives Include:

- **Kinematics & Mobility:** Designing a stable, differentially driven mobile platform capable of smooth motion control and payload management.
- **Autonomous Navigation Logic:** Developing reliable closed-loop control algorithms for line-following using multi-array IR sensors.
- **Sensor Fusion & Safety:** Integrating HC-SR04 ultrasonic sensors to create a fail-safe obstacle detection system, preventing collisions in a dynamic warehouse environment.
- **Smart Inventory Management:** Interfacing RFID (RC522) and vision modules to detect, recognize, and log inventory data in real time.
- **Embedded Systems Integration:** Wiring, soldering, and programming the entire hardware ecosystem using embedded C/C++ on the Arduino platform.

3. System Architecture & Hardware Selection

To meet the project requirements, our team carefully selected and validated the following hardware components:

- I. **Core Processing: Arduino UNO** – Selected for its robust GPIO capabilities and extensive library support for sensor integration.
- II. **Actuation & Drive System:**
 - **DC Gear Motors (100–300 RPM)** – Providing the necessary torque for warehouse navigation.
 - **Motor Driver (L298N / L293D):** Crucial for handling the high-current demands of the motors while allowing bidirectional speed control via PWM signals.
- III. **Chassis:** Custom-fabricated base supported by 2 main drive wheels and 1 passive caster wheel for zero-radius turning capability.



IV. Navigation & Safety:

- **5-Array IR Line Sensor** – Chosen over a single sensor for higher resolution path-tracking and smoother PID control.
- **HC-SR04 Ultrasonic Sensor** – Provides non-contact distance measurement for obstacle avoidance.

V. **Data Logging & Identification: RC522 RFID Reader Module & Arduino Camera Module** – Working in tandem to scan physical tags and visually verify warehouse items.

VI. **Power Distribution:** A unified power management system utilizing a **Buck Converter / Voltage Regulator** to safely step down voltage for sensitive logic components, alongside a master DC Power Switch.

4. Software Implementation Source Code

The following embedded C/C++ code was developed and deployed on the Arduino UNO during Phase 6. The software architecture is designed around a non-blocking main `loop()` that prioritizes obstacle detection via the HC-SR04 ultrasonic sensor. If the path is clear, the system defaults to the `followLine()` routine. Upon detecting an obstacle (inventory item) within the predefined threshold, the system halts actuation and triggers the `scanInventory()` interrupt to read and log the RC522 RFID UID to the serial database.

```
#include <SPI.h>
#include <MFRC522.h>

// --- 1. PIN DEFINITIONS ---
// Motors
const int enA = 6;
const int in1 = 8;
const int in2 = 7;
const int in3 = 4;
const int in4 = 2;
const int enB = 5;
```

```

// Sensors
const int leftSensor = A0;
const int rightSensor = A1;
const int trigPin = A2;
const int echoPin = A3;

// RFID
#define SS_PIN 10
#define RST_PIN 9
MFRC522 mfrc522(SS_PIN, RST_PIN);

// --- 2. SETTINGS ---
int motorSpeed = 80; // Speed (0-255). Adjust if too fast/slow.
int stopDistance = 4; // Stop if object is closer than threshold

void setup() {
  Serial.begin(9600); // Start Serial Monitor (Your Database)

  // Motor Pins
  pinMode(enA, OUTPUT); pinMode(in1, OUTPUT); pinMode(in2, OUTPUT);
  pinMode(enB, OUTPUT); pinMode(in3, OUTPUT); pinMode(in4, OUTPUT);

  // Sensor Pins
  pinMode(leftSensor, INPUT);
  pinMode(rightSensor, INPUT);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  // RFID Init
  SPI.begin();
  mfrc522.PCD_Init();

  Serial.println("SYSTEM ONLINE: Placing Robot on Track...");
  delay(2000); // 2 second delay to let you put the robot down
}

void loop() {
  // STEP 1: Check for Obstacles (Boxes)
  int distance = getDistance();

  if (distance > 0 && distance < stopDistance) {
    // OBSTACLE DETECTED!
    stopMotors();
  }
}

```

```

    Serial.println("OBSTACLE: Scanning for Inventory...");
    scanInventory();
}
else {
    // PATH CLEAR -> Follow the Line
    followLine();
}
}

// --- LOGIC FUNCTIONS ---

void followLine() {
    // Read IR Sensors (Assuming 1 = Black Line, 0 = White Surface)
    // If your sensors are opposite, swap 1 and 0 here.
    int leftVal = digitalRead(leftSensor);
    int rightVal = digitalRead(rightSensor);

    // CASE 1: Forward (Line in middle / Both White)
    if (leftVal == 0 && rightVal == 0) {
        moveForward();
    }
    // CASE 2: Turn Left (Right sensor lost line / Left sensor on line)
    else if (leftVal == 1 && rightVal == 0) {
        turnLeft();
    }
    // CASE 3: Turn Right (Left sensor lost line / Right sensor on line)
    else if (leftVal == 0 && rightVal == 1) {
        turnRight();
    }
    // CASE 4: Stop (Both on Black - Intersection/End)
    else if (leftVal == 1 && rightVal == 1) {
        stopMotors();
    }
}

void scanInventory() {
    // Check if a card is near the reader
    if ( ! mfrc522.PICC_IsNewCardPresent()) return;
    if ( ! mfrc522.PICC_ReadCardSerial()) return;

    // Print UID to Serial Monitor
    Serial.print(">> INVENTORY LOGGED: Item ID ");
    for (byte i = 0; i < mfrc522.uid.size; i++) {
        Serial.print(mfrc522.uid.uidByte[i] < 0x10 ? " 0" : " ");
    }
}

```

```

    Serial.print(mfrc522.uid.uidByte[i], HEX);
}
Serial.println();

// Pause to prevent reading the same tag 100 times in 1 second
delay(2000);

// Stop reading this specific card
mfrc522.PICC_HaltA();
}

int getDistance() {
    digitalWrite(trigPin, LOW); delayMicroseconds(2);
    digitalWrite(trigPin, HIGH); delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    long duration = pulseIn(echoPin, HIGH);
    return duration * 0.034 / 2;
}

// --- MOTOR FUNCTIONS ---

void moveForward() {
    analogWrite(enA, motorSpeed);
    analogWrite(enB, motorSpeed);
    digitalWrite(in1, HIGH); digitalWrite(in2, LOW);
    digitalWrite(in3, HIGH); digitalWrite(in4, LOW);
}

void turnLeft() {
    analogWrite(enA, motorSpeed + 20); // Give a little kick to turn
    analogWrite(enB, motorSpeed + 20);
    digitalWrite(in1, LOW); digitalWrite(in2, HIGH); // Left Back
    digitalWrite(in3, HIGH); digitalWrite(in4, LOW); // Right Fwd
}

void turnRight() {
    analogWrite(enA, motorSpeed + 20);
    analogWrite(enB, motorSpeed + 20);
    digitalWrite(in1, HIGH); digitalWrite(in2, LOW); // Left Fwd
    digitalWrite(in3, LOW); digitalWrite(in4, HIGH); // Right Back
}

void stopMotors() {
    digitalWrite(in1, LOW); digitalWrite(in2, LOW);

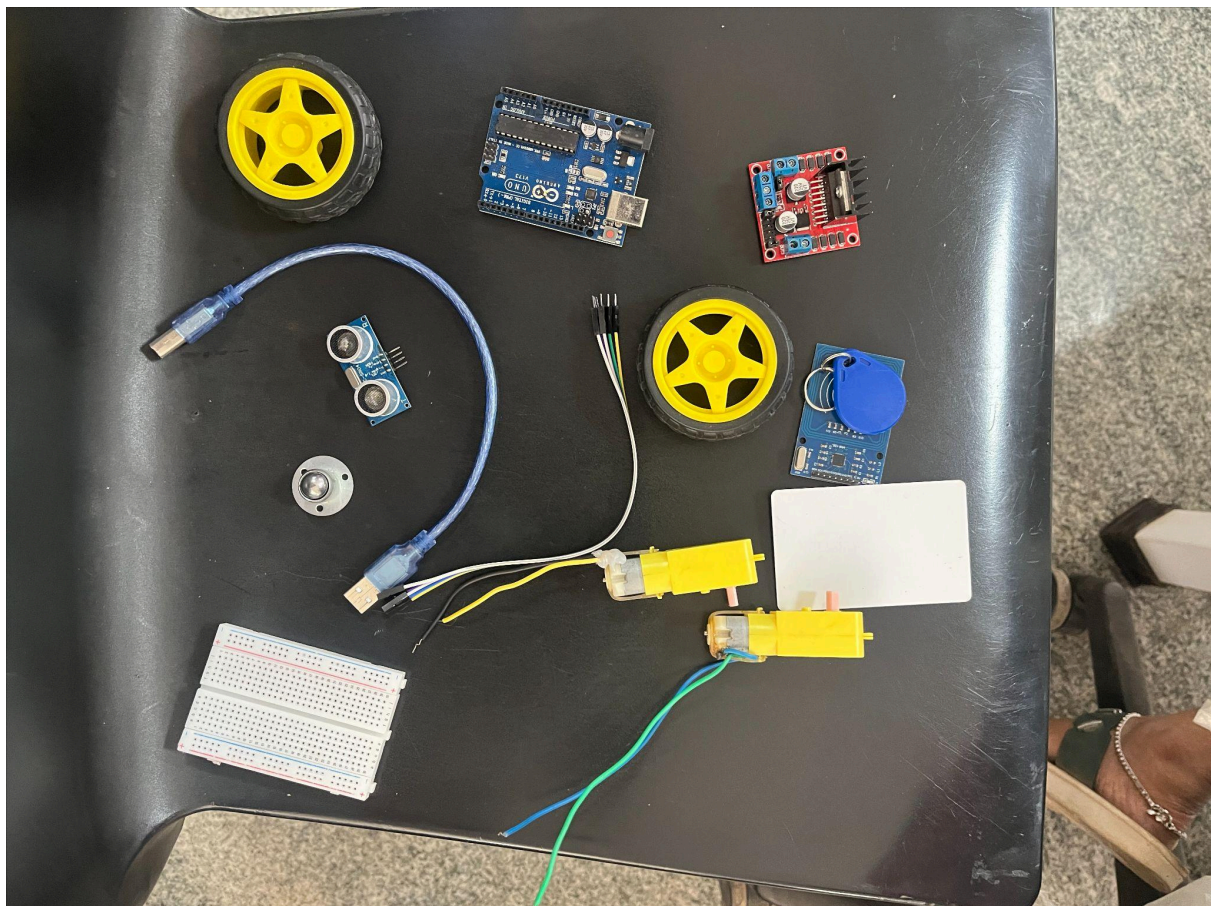
```

```
digitalWrite(in3, LOW); digitalWrite(in4, LOW);  
}
```

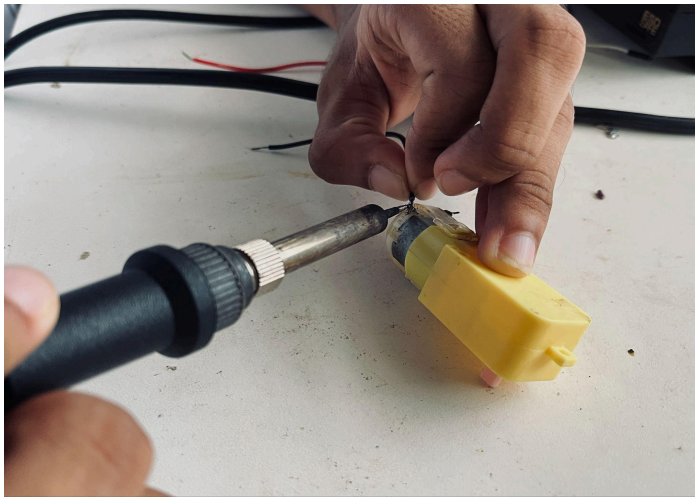
5. Implementation Log & Milestone Tracking

The project has been executed in phased milestones, moving from initial procurement to physical assembly.

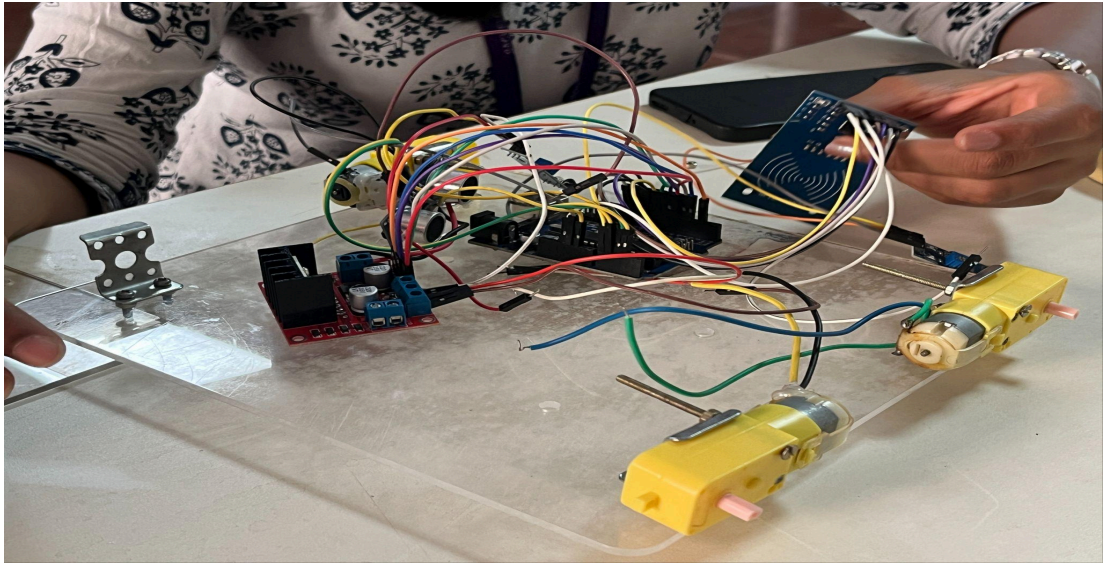
- **Phase 1:** Component Procurement & Diagnostics (**February 2**): The project commenced with the collection of all required electronic and mechanical hardware from the TCE lab. Our initial focus was on quality assurance; we conducted bench tests on individual modules (IR arrays, motor drivers, and RFID readers) to verify functionality before integration.



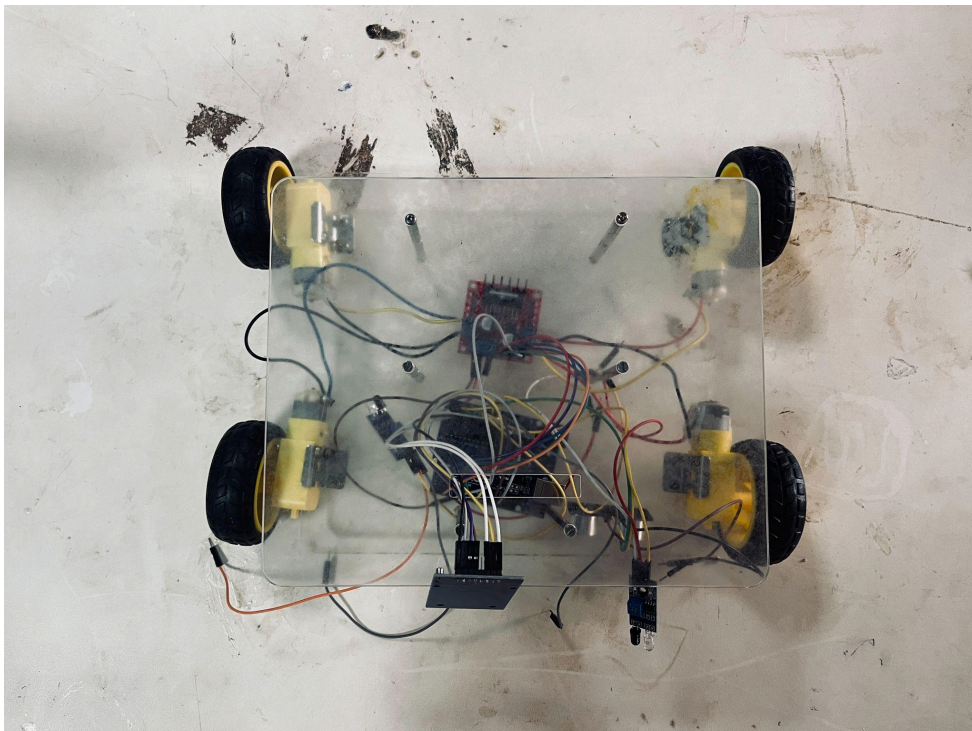
- **Phase 2: Architectural Planning & Initial Design (February 5):** The team developed the schematic wiring diagrams and finalized the digital CAD model for the robot's physical chassis. This phase involved calculating the optimal placement for sensors to ensure an unobstructed field of view.
- **Phase 3: Fabrication & Iterative Problem Solving (February 12 & 20):** Initial manufacturing took place in the Makerspace, where the chassis was cut and prepared. During the preliminary physical assembly on Feb 12, we identified a dimensional error—a missing mounting hole on the top chassis plate. Demonstrating rapid prototyping principles, the team revised the design files and manufactured a newly corrected chassis on Feb 20. We simultaneously procured the necessary mechanical standoffs, spacers, and structural fasteners (nuts/bolts) from the TCE lab.



- **Phase 4: Mechanical Integration & Soldering (February 23):** The mechanical build was successfully finalized. The newly fabricated chassis was assembled, and all major sub-systems were securely mounted. To ensure electrical reliability during movement, all motor terminals were permanently soldered, mitigating the risk of loose connections during the upcoming dynamic testing.

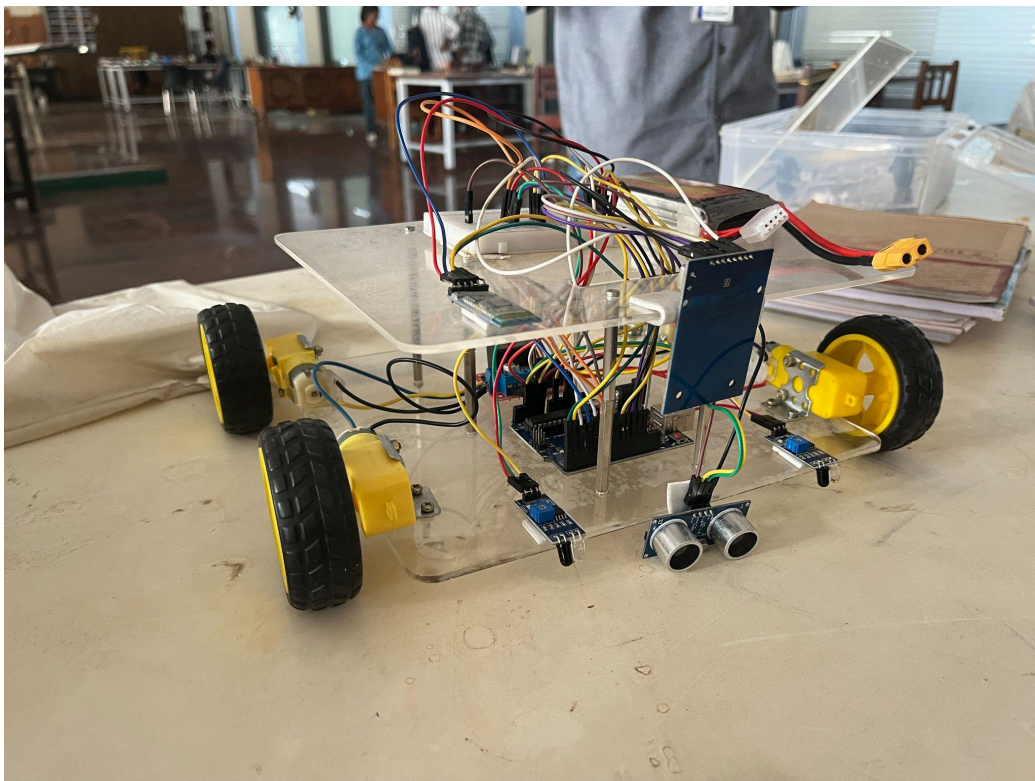


- **Phase 5: Final electric connection (February 26):** which involved routing the power lines from the battery through the buck converter to both the Arduino and the motor driver, as well as securely interfacing the logic pins of the L298N, IR array, HC-SR04, and RC522 to the Arduino UNO using secure jumper configurations.



- **Phase 6: Embedded Programming & Logic Integration (February 28)** During this crucial software phase, which culminated on the February 28 deadline, the team focused extensively on embedded programming and system logic integration. We

developed the primary navigation loop by writing robust C/C++ control algorithms for the Arduino UNO, allowing it to interpret real-time data from the 5-array IR sensor and execute smooth, continuous line-following maneuvers. Simultaneously, we engineered efficient interrupt service routines (ISRs) and polling mechanisms to interface with the RC522 RFID module and the Arduino Camera. This non-blocking software architecture ensured that the robot could detect, scan, and log inventory data seamlessly in the background without causing processing delays or halting its physical movement along the simulated warehouse track.



Phase 7: Real-World Testing & Calibration (Completed February 1) During this final validation phase, the fully assembled robot was deployed onto a simulated warehouse test track to evaluate its physical performance and sensory accuracy in a real-world environment. The team meticulously tuned the multi-array infrared (IR) sensor thresholds to account for varying ambient light conditions, ensuring robust and consistent line detection regardless of external lighting interference. Additionally, we iteratively adjusted the Pulse Width Modulation (PWM) values sent to the L298N motor driver. This precise calibration allowed us to fine-tune the differential drive motor speeds, resulting in smooth, jitter-free navigation as the autonomous system dynamically traversed its predefined warehouse paths.

6. Challenges, Risks & Mitigation Strategies

As we move into the final phases of development, the team has identified potential risks and established mitigation strategies to ensure we meet the **February 28** deadline:

- I. **Risk:** Time-consuming PID (Proportional-Integral-Derivative) tuning for the line-following algorithm causing project delays.
 - A. **Mitigation:** We will begin with simple threshold-based logic to ensure basic movement, then incrementally introduce proportional control to smooth out the turning, rather than attempting complex PID tuning all at once.
- II. **Risk:** Power fluctuations causing the Arduino to reset when motors draw stall current.
 - A. **Mitigation:** We are isolating the motor power supply from the logic power supply using the buck converter and ensuring common ground connections are secure.
- III. **Risk:** Sensor interference between the IR array and ambient warehouse lighting.
 - A. **Mitigation:** The IR sensors will be mounted as close to the floor as physically possible, and we will code a dynamic calibration routine that reads ambient light levels upon startup.

7. Conclusion

The Warehouse Line-Following & Inventory Robot project was successfully completed and deployed on March 1, 2026, under the guidance of Pulkit Sir. Our team successfully engineered a fully functional autonomous system capable of navigating predefined warehouse paths, detecting obstacles, and logging inventory in real-time using RFID technology. This project bridged the gap between theoretical robotics and practical industrial automation, providing invaluable hands-on experience in embedded C programming, sensor integration, and electro-mechanical troubleshooting. Ultimately, the successful prototype demonstrates a strong foundation for developing smart logistics solutions in an Industry 4.0 environment.